Evaluation of Hardware Designs. Resource Utilization

1 Purpose

A hardware design implemented on an FPGA device is characterized by three metrics: the amount of FPGA **resources** it uses, the **power** it consumes and the **maximum frequency** (F_{max}) of its clock. This laboratory work presents how a design is deployed on a board and how its resource utilization is evaluated from the post-implementation reports of Vivado.

The evaluated design is the module built in the exercises of the previous laboratory work (the sum of a window of pre-saturated values). It is deployed on the Digilent Basys3 board [4] by means of a simple wrapper, which is created in this laboratory work.

2 FPGA Resources

The logic of a 7-series FPGA, such as the Artix-7 XC7A35T device on the Basys3 board, is organized in *Configurable Logic Blocks* (CLBs), each containing two *slices*. A slice contains four 6-input Look-Up Tables (LUTs), eight Flip-Flops (FFs) and a fast carry chain used to implement adders and comparators. Some of the LUTs can also be used as small distributed memories (LUTRAM) or shift registers (SRL). Besides the slices, the device contains dedicated blocks: Block RAMs (BRAM), DSP slices (multiply-accumulate units), clocking resources and I/O blocks (IOB). Table 1 shows the resources available on the XC7A35T device.

Table 1: Resources of the Artix-7 XC7A35T-1CPG236C device (Basys3)

Resource	**Available**
Slice LUTs	20800
Slice registers (FFs)	41600
Block RAM tiles (36 Kb)	50
DSP slices	90
IOBs	106

The resource utilization is the number of resources of each type used by the design after implementation (after place and route stage), usually also expressed as a percentage of the available resources. It indicates the cost of the design expressed in terms of how large a device it needs, or how many instances of it fit on a given device.

3 Design Under Evaluation

The evaluated design (Design Under Test – DUT) is the `saturated_window_adder` implemented in exercise 3 of the previous laboratory work: an AXI4-Stream saturator connected to an AXI4-Stream sliding window adder. Use your own implementation, verified in the previous laboratory work (`saturator.vhd`, `sliding_window_adder.vhd` and `saturated_window_adder.vhd`). The wrapper described in Section 4 assumes the following interface:

```
entity saturated_window_adder is
  Generic (
    WINDOW_SIZE : integer := 5
  );
  Port (
    aclk : IN STD_LOGIC;
    aresetn : IN STD_LOGIC;
    s_axis_val_tvalid : IN STD_LOGIC;
    s_axis_val_tready : OUT STD_LOGIC;
    s_axis_val_tdata : IN STD_LOGIC_VECTOR(31 DOWNTO 0);
    s_axis_max_tvalid : IN STD_LOGIC;
    s_axis_max_tready : OUT STD_LOGIC;
    s_axis_max_tdata : IN STD_LOGIC_VECTOR(31 DOWNTO 0);
    s_axis_min_tvalid : IN STD_LOGIC;
    s_axis_min_tready : OUT STD_LOGIC;
    s_axis_min_tdata : IN STD_LOGIC_VECTOR(31 DOWNTO 0);
    m_axis_sum_tvalid : OUT STD_LOGIC;
    m_axis_sum_tready : IN STD_LOGIC;
    m_axis_sum_tdata : OUT STD_LOGIC_VECTOR(31 DOWNTO 0)
  );
end saturated_window_adder;
```

The DUT must work correctly, as verified by simulation in the previous laboratory work.

4 Creating the Basys3 Wrapper

4.1 Why a Wrapper Is Needed

The design cannot be implemented directly on the board because the DUT has 102 input bits (`aclk`, `aresetn`, three 32-bit **TDATA** inputs, three input **TVALID** signals and the output **TREADY**) and 36 output bits, i.e. 138 ports, while the device has only 106 IOBs, and the board connects only a few buttons, switches, LEDs and a 4-digit seven-segment display to them.

Therefore, the DUT has to be placed in a simple *wrapper*, `basys3_top` (see Figure 1), which provides the inputs of the DUT and shows its result on the board. The wrapper works as follows (also see Table 2):

- the input values *val*, *min* and *max* are hardcoded: they are constants connected directly to the **TDATA** inputs of the DUT;
- each press of `btn(0)` (the center button) makes the three inputs valid, for one transfer;
- the seven-segment display shows bits 15..0 of the sum, in hexadecimal, and the LEDs show bits 31..16;
- `btn(1)` (the up button) resets the DUT.

Table 2: Board interface of the wrapper

Board I/O	Function
`clk`	100 MHz clock
`btn(0)` (center)	sends the hardcoded values to the DUT, once
`btn(1)` (up)	resets the DUT (the window and the sum are cleared)
display (`cat`, `an`)	bits 15..0 of the sum, in hexadecimal
`led(15 downto 0)`	bits 31..16 of the sum

The debouncer and the 7-segment display modules are provided, together with the constraints file `basys3_constr.xdc`, in the folder of this laboratory work (their content is also listed in Appendix A):

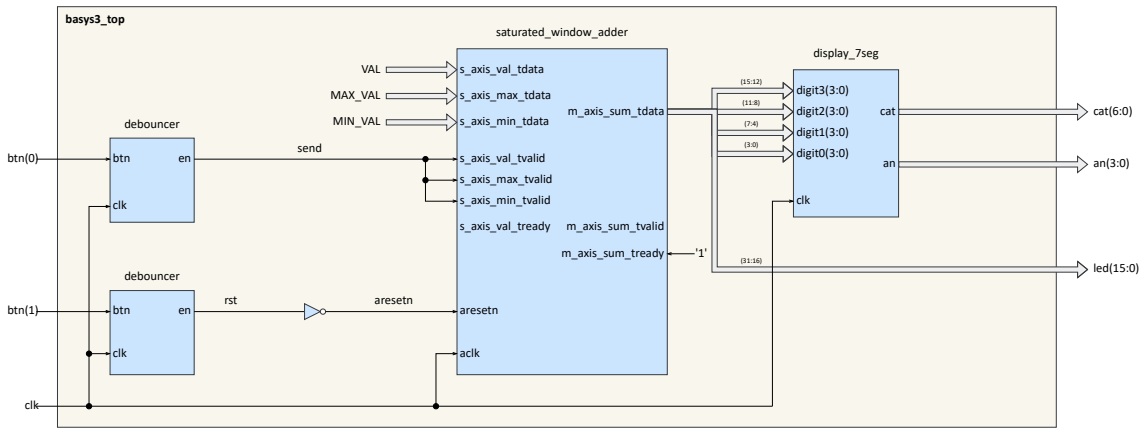


Figure 1: The Basys3 wrapper of the evaluated design

- `debouncer`: samples the button `btn` every 2^{16} clock cycles and generates a pulse of one clock cycle for each press;
- `display_7seg`: shows the four hexadecimal digits `digit3` ... `digit0` on the seven-segment display (`cat` are the active-LOW segments, `an` the active-LOW anodes).

4.2 Implementing the Wrapper

Write the wrapper in a new file, `basys3_top.vhd`, following the points below.

1. The entity. The port names must match the names used in the provided constraints file, `basys3_constr.xdc`:

```
entity basys3_top is
  Port ( clk : in STD_LOGIC;
         btn : in STD_LOGIC_VECTOR (4 downto 0);
         sw : in STD_LOGIC_VECTOR (15 downto 0);
         led : out STD_LOGIC_VECTOR (15 downto 0);
         an : out STD_LOGIC_VECTOR (3 downto 0);
         cat : out STD_LOGIC_VECTOR (6 downto 0));
end basys3_top;
```

The five buttons are mapped in `basys3_constr.xdc` as follows: `btn(0)` center, `btn(1)` up, `btn(2)` left, `btn(3)` right, `btn(4)` down. The wrapper uses only `btn(0)` and `btn(1)`; the switches are not used.

2. The components. In the declarative part of the architecture, declare the three components used by the wrapper: `saturated_window_adder` (including its generic), `debouncer` and `display_7seg`. Each component declaration repeats the generics and the ports of the corresponding entity. The components are instantiated by name (e.g. `send_btn : debouncer port map (...)`), so the wrapper does not refer to the `work` library.

3. The buttons. Instantiate the `debouncer` twice: `send_btn` for `btn(0)`, producing the signal `send`, and `rst_btn` for `btn(1)`, producing the signal `rst`. The active-LOW reset of the DUT is the negated `rst` signal, `aresetn <= not rst;` (a concurrent assignment): since the debouncer generates a pulse of one clock cycle, `aresetn` is LOW for one clock cycle when `btn(1)` is pressed, which is enough for the synchronous reset of the DUT.

4. The window size and the input values. The window size and the input values are hardcoded as constants, declared in the declarative part of the architecture. The window size is passed to the `WINDOW_SIZE` generic of the DUT, and the input values are connected directly to the **TDATA** inputs of the DUT. The values are written directly as 32-bit hexadecimal literals (negative values in two's complement), for example $val = 20$, $min = -10$ and $max = 10$:

```
constant WINDOW_SIZE : integer := 5;
constant VAL : STD_LOGIC_VECTOR(31 downto 0) := x"00000014";      -- 20
constant MIN_VAL : STD_LOGIC_VECTOR(31 downto 0) := x"FFFFFFF6"; -- -10
constant MAX_VAL : STD_LOGIC_VECTOR(31 downto 0) := x"0000000A"; -- 10
```

To evaluate the DUT for another window size or with other input values, change the constants and implement the design again.

5. The handshake signals. The `send` signal is connected directly to the three **TVALID** inputs of the DUT, so the three values are valid together, for one clock cycle, at each press of `btn(0)`. The output **TREADY**, `m_axis_sum_tready`, is tied to `'1'` so that the result is always accepted. Therefore the DUT is idle, and ready to consume new inputs, whenever a button is pressed, and the handshake takes place in the clock cycle in which `send` is HIGH. The **TREADY** outputs of the input interfaces and the **TVALID** output, `m_axis_sum_tvalid`, can be either left unconnected, or connected to some dummy signals.

6. The DUT. Instantiate `saturated_window_adder` with the label `dut_inst`, map its `WINDOW_SIZE` generic to the constant of point 4 and connect the three **TDATA** inputs to the constants of point 4, the three **TVALID** inputs to `send`, `m_axis_sum_tready` to `'1'` and `m_axis_sum_tdata` to a 32-bit signal `sum`.

Then add the `dont_touch` attribute [2] for the DUT instance. The attribute is declared and assigned in the **declarative part of the architecture** of `basys3_top` (between `architecture ... is` and `begin`, together with the component, constant and signal declarations), and it refers to the instance by its label, `dut_inst`, which must be the same label as the one used when the DUT is instantiated in the body of the architecture:

```
architecture Behavioral of basys3_top is

    -- component declarations, constants, signals
    ...

    attribute dont_touch : string;
    attribute dont_touch of dut_inst : label is "true";

begin

    ...

    dut_inst : saturated_window_adder        -- the label used in the attribute
        generic map (WINDOW_SIZE => WINDOW_SIZE)
        port map ( ... );

    ...

end Behavioral;
```

Since the input values and the output **TREADY** are constants, the synthesis tool would otherwise compute the result from them and remove most of the logic of the DUT. With `dont_touch`, the DUT is implemented completely, as it would be in any other system.

7. The outputs. Instantiate `display_7seg` and connect its four digits to bits 15..0 of `sum` (`digit0` to `sum(3 downto 0)`, ..., `digit3` to `sum(15 downto 12)`) and its `cat` and `an` outputs to the ports of the wrapper. Connect bits 31..16 of `sum` to the LEDs.

4.3 Constraints

The provided file `basys3_constr.xdc` assigns the pins of the Basys3 board to the ports `clk`, `sw`, `led`, `cat`, `an` and `btn` of the wrapper. It also contains the clock constraint, without which Vivado would not know the frequency of the clock and would not analyze the timing of the design:

```
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
```

The constraint states that `clk` is a 100 MHz clock (a period of 10 ns). No timing constraints are set for the other ports: the buttons, the switches, the LEDs and the display change at human speed, asynchronously to the clock, so the paths through these ports are not timed (Vivado reports them as unconstrained) and only the internal register-to-register paths are analyzed.

5 Implementation Flow in Vivado

This section provides a guideline on the steps needed to create the Vivado project of the design, to run its synthesis and implementation and to obtain the post-implementation reports.

Step 1. Create a new RTL project using Vivado IDE targeting the Basys 3 device

Open Vivado, click on *File* → *Project* → *New...* and in the opened window click *Next*. Provide the name of the project and the path or browse to the directory where you want to create the project and click *Next*. In the *Project Type* window select *RTL Project* and tick the *Do not specify sources at this time* option. The device properties that you need to choose are:

- **Product category:** All
- **Family:** Artix-7
- **Package:** cpg236
- **Speed grade:** -1

Then select from the table the **xc7a35tcpg236-1** device and click *Next*. Click *Finish*.

Step 2. Add the sources of the DUT

In the *Sources* tab, right-click on *Design Sources* and then click on *Add Sources...*. Select *Add or create design sources* and press *Next*. Click on *Add Files*, navigate to the location of the files of your `saturated_window_adder` from the previous laboratory work (`saturator.vhd`, `sliding_window_adder.vhd` and `saturated_window_adder.vhd`), select them and press *OK*. Check that *Copy sources into project* is checked and then press *Finish*.

Step 3. Include the debouncing circuit and the 7-segment display module

In the *Sources* tab, right-click on *Design Sources* and then click on *Add Sources...*. Select *Add or create design sources* and press *Next*. Click on *Add Files*. In the navigation window select for the *Files of type* field the value *All Files* and navigate to the location of the provided `debouncer.vhd` and `display_7seg.vhd` files on your computer. Select them, press *OK*, check that *Copy sources into project* is checked and then press *Finish*.

Step 4. Create the top level design targeting the Basys 3 device

In the *Sources* tab, right-click on *Design Sources* and then click on *Add Sources...*. Select *Add or create design sources* and click on *Next*. Then select *Create File*, type `basys3_top` in the *File name* text field, make sure that the selected *File type* is VHDL and press *OK*. Then, click on *Finish*. In the *Define Module* window you can leave the ports undefined and press *OK*, since they are defined manually in the created `.vhd` file.

Write the wrapper in the `basys3_top.vhd` file, as described in Section 4. After implementing the top level entity, right-click on the `basys3_top.vhd` file within the *Design Sources* section of the *Sources* tab and select *Set as Top*.

Step 5. Import the Basys 3 `.xdc` constraints file

In the *Sources* tab, right-click on *Constraints* and then click on *Add Sources...*. Select *Add or create constraints* and press *Next*. Click on *Add Files*. Navigate to the location of the provided

`basys3_constr.xdc` file on your computer. After finding the file, select it and press *OK*. Check that *Copy constraints files into project* is checked and then press *Finish*. Right-click on the file in the *Constraints* section of the *Sources* tab and select *Set as Target Constraint File*. The target constraints file is the file in which Vivado saves the constraints created or edited from the GUI. Open the file and check that it contains the clock constraint described in Section 4.3, with a period of 10 ns.

Step 6. Configure the project settings

In the *Project Manager* menu of the *Flow Navigator* click on *Settings* and set:

- in *Synthesis*: the *Vivado Synthesis Defaults* strategy, and *Incremental synthesis* disabled, so that each implementation starts from scratch;
- in *Implementation*: the *Performance_Explore* strategy (the *Explore* directive for every implementation step, as recommended by AMD for performance evaluations [3]).

Press *OK*.

Step 7. Run Synthesis and Implementation

In the *Synthesis* menu of the *Flow Navigator* click on *Run Synthesis* and in the newly opened window press *OK*. When the *Synthesis Completed* window appears, click on *Cancel*. Check the *Messages* tab for errors and critical warnings.

Similarly, in the *Implementation* menu of the *Flow Navigator* click on *Run Implementation* and in the newly opened window press *OK*. When the *Implementation Completed* window appears, also click on *Cancel*.

During the implementation run, Vivado automatically generates a set of reports, listed in the *Reports* tab (*Window* → *Reports*), under *Implementation*: among others, *Utilization - Place Design*, *Power - Route Design* and *Timing Summary - Route Design* (`basys3_top_utilization_placed.rpt`, `basys3_top_power_routed.rpt` and `basys3_top_timing_summary_routed.rpt` files that can be found in the `<project_name>.runs/impl_1` sub-directory of your project). The main values are also shown in the *Project Summary* (*Utilization*, *Power* and *Timing* panels) and in the *Design Runs* tab. The detailed, interactive reports are obtained on the implemented design (*Flow Navigator* → *Open Implemented Design*), as described in the next section.

6 Evaluating the Resource Utilization

The resource utilization of the implemented design is obtained with the `report_utilization` command [1], which can be run from the GUI:

1. Open the implemented design and generate the resource utilization report as follows: *Flow Navigator* → *Implementation* → *Open Implemented Design* → *Report Utilization...* (also available as *Reports* → *Report Utilization...*). Keep the default options and press *OK*. The equivalent Tcl command is printed in the Tcl console: `report_utilization -name utilization_1`.
2. The *Utilization* window opens. The left panel contains the *Summary* and the tables of the report: *Slice Logic* (LUTs, divided in LUTs used as logic and as memory, and registers), *Slice Logic Distribution* (the occupied slices), *Memory* (BRAMs), *DSP*, *IO and GT Specific* (IOBs) and *Clocking* (`BUFG` and MMCM/PLL).
3. Select *Hierarchy* in the left panel to see the resources of every cell: expand `basys3_top` and `dut_inst` to read the LUTs and FFs of the saturator and of the sliding window adder instances (listed under the instance names used in your `saturated_window_adder`). Selecting a cell in this table highlights its resources in the *Device* view, which shows where the DUT is placed on the chip.
4. To save the report as a text file, use *File* → *Export* → *Export Report* or the Tcl command `report_utilization -hierarchical -file util_hier.rpt`.

The report generated automatically by the implementation run (*Utilization - Place Design*) is created after placement, but the utilization does not change during routing, so it contains the same values. It does not contain the hierarchical table, which is obtained only interactively or with the `-hierarchical` option.

In the *Hierarchy* view, the cells other than `dut_inst` (the debouncers, the display driver and the top level) show the resources added by the wrapper, which are not part of the evaluated design. Note also that the LUTs of a cell may not add up exactly to the sum of its sub-cells, since a physical LUT can implement two functions belonging to different cells.

7 Exercises

1. **Implement the Basys3 wrapper of the `saturated_window_adder` and run the synthesis and the implementation of the design.**

   Design and Implementation Considerations:

   - Take your `saturated_window_adder` from the previous laboratory work (Section 3) and write the wrapper, `basys3_top.vhd`, following the steps of Section 4, using the provided `debouncer` and `display_7seg` modules.
   - Create the project as described in Section 5, adding the provided `basys3_constr.xdc` and setting it as the target constraints file.
   - Run the synthesis and the implementation for `WINDOW_SIZE` $= 5$ and fix the errors and critical warnings, if any.
   - Open the implemented design and generate the hierarchical utilization report (Section 6). Record the number of LUTs, FFs, BRAMs and DSPs used by `dut_inst` and by its saturator and sliding window adder instances in the row for `WINDOW_SIZE` $= 5$ of Table 3 (see Exercise 2).

2. **Evaluate the resource utilization of the DUT as a function of the window size.**

   Considerations:

   - Implement the design for `WINDOW_SIZE` $\in \{10, 20, 30, 40, 50, 60, 70\}$ by changing the value of `WINDOW_SIZE` constant in `basys3_top.vhd`, keep the 10 ns clock constraint and re-run the implementation.
   - For each window size, read the number of LUTs, FFs, BRAMs and DSPs used by `dut_inst` and by its saturator and sliding window adder instances from the *Hierarchy* view of the utilization report and record them in a table such as Table 3, with one row for each value of `WINDOW_SIZE`.

Table 3: Template for recording the resource utilization as a function of the window size

`WINDOW`	**DUT** (`dut_inst`)				**saturator**				**sliding window adder**			
`SIZE`	LUT	FF	BRAM	DSP	LUT	FF	BRAM	DSP	LUT	FF	BRAM	DSP
10												
20												
30												
40												
50												
60												
70												

   - Plot the number of LUTs and the number of FFs as a function of `WINDOW_SIZE`, with one curve for the DUT, one for the saturator and one for the sliding window adder, as in the template of Figure 2. The plots can be drawn with any tool (e.g. a spreadsheet, Python or `pgfplots`).

- Explain how each resource varies with the window size an try to derive a formula for the number of FFs of the DUT as a function of `WINDOW_SIZE`, by comparing the number of flip-flops with the registers of your VHDL description, and check it against the recorded values. Explain how the number of LUTs grows and why the resources of the saturator do not change.

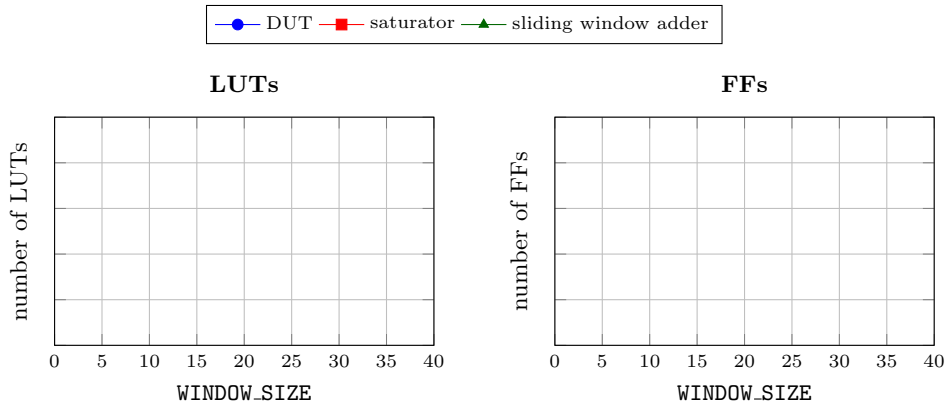


Figure 2: Template for plotting the resource utilization as a function of the window size

References

[1] Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906) - AMD [accessed Oct. 2026], https://docs.amd.com/r/en-US/ug906-vivado-design-analysis

[2] Vivado Design Suite User Guide: Synthesis (UG901) - AMD [accessed Oct. 2026], https://docs.amd.com/r/en-US/ug901-vivado-synthesis

[3] UltraFast Design Methodology Guide for FPGAs and SoCs (UG949) - AMD [accessed Oct. 2026], https://docs.amd.com/r/en-US/ug949-vivado-design-methodology

[4] Basys 3 Reference Manual - Digilent [accessed Oct. 2026], https://digilent.com/reference/programmable-logic/basys-3/reference-manual

A Code appendix

A.1 Debouncer Circuit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity debouncer is
  Port ( clk : in std_logic;
         btn : in std_logic;
         en : out std_logic );
end debouncer;

architecture Behavioral of debouncer is

signal count : std_logic_vector(15 downto 0) := (others => '0');
signal q1 : std_logic := '0';
signal q2 : std_logic := '0';
signal q3 : std_logic := '0';

begin

    process(clk)
```

```
    begin
        if rising_edge(clk) then
            count <= count + 1;
        end if;
    end process;

    process(clk)
    begin
        if rising_edge(clk) then
            if count(15 downto 0) = x"1111" then
                q1 <= btn;
            end if;
        end if;
    end process;

    process(clk)
    begin
        if rising_edge(clk) then
            q2 <= q1;
            q3 <= q2;
        end if;
    end process;

    en <= q2 and (not q3);

end Behavioral;
```

A.2 7-Segment Display

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity display_7seg is
    Port ( digit0 : in STD_LOGIC_VECTOR (3 downto 0);
           digit1 : in STD_LOGIC_VECTOR (3 downto 0);
           digit2 : in STD_LOGIC_VECTOR (3 downto 0);
           digit3 : in STD_LOGIC_VECTOR (3 downto 0);
           clk : in STD_LOGIC;
           cat : out STD_LOGIC_VECTOR (6 downto 0);
           an : out STD_LOGIC_VECTOR (3 downto 0));
end display_7seg;

architecture Behavioral of display_7seg is

signal cnt : STD_LOGIC_VECTOR (15 downto 0) := (others => '0');
signal digit_to_display : STD_LOGIC_VECTOR (3 downto 0) := (others => '0');

begin

    process (clk)
    begin
        if rising_edge(clk) then
            cnt <= cnt + 1;
        end if;
    end process;

    digit_to_display <= digit0 when cnt(15 downto 14) = "00" else
                        digit1 when cnt(15 downto 14) = "01" else
                        digit2 when cnt(15 downto 14) = "10" else
                        digit3;

    an <= "1110" when cnt(15 downto 14) = "00" else
          "1101" when cnt(15 downto 14) = "01" else
          "1011" when cnt(15 downto 14) = "10" else
          "0111";

    -- HEX-to-seven-segment decoder
          --    HEX:    in    STD_LOGIC_VECTOR (3 downto 0);
          --    LED:    out   STD_LOGIC_VECTOR (6 downto 0);
          --
```

```
        -- segment encoinputg
        --      0
        --     ---
        --  5 |   | 1
        --     ---   <- 6
        --  4 |   | 2
        --     ---
        --      3

    with digit_to_display select
         cat <= "1111001" when "0001",   --1
                "0100100" when "0010",   --2
                "0110000" when "0011",   --3
                "0011001" when "0100",   --4
                "0010010" when "0101",   --5
                "0000010" when "0110",   --6
                "1111000" when "0111",   --7
                "0000000" when "1000",   --8
                "0010000" when "1001",   --9
                "0001000" when "1010",   --A
                "0000011" when "1011",   --b
                "1000110" when "1100",   --C
                "0100001" when "1101",   --d
                "0000110" when "1110",   --E
                "0001110" when "1111",   --F
                "1000000" when others;   --0

end Behavioral;
```

A.3 Basys 3 .xdc constraints file

```
# This file is a general .xdc for the Basys3 rev B board
# To use it in a project:
# - uncomment the lines corresponding to used pins
# - rename the used ports (in each line, after get_ports) according to the top level signal
# names in the project

# Clock signal
set_property PACKAGE_PIN W5 [get_ports clk]
 set_property IOSTANDARD LVCMOS33 [get_ports clk]
 create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]

# Switches
set_property PACKAGE_PIN V17 [get_ports {sw[0]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
set_property PACKAGE_PIN V16 [get_ports {sw[1]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
set_property PACKAGE_PIN W16 [get_ports {sw[2]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
set_property PACKAGE_PIN W17 [get_ports {sw[3]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
set_property PACKAGE_PIN W15 [get_ports {sw[4]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
set_property PACKAGE_PIN V15 [get_ports {sw[5]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
set_property PACKAGE_PIN W14 [get_ports {sw[6]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
set_property PACKAGE_PIN W13 [get_ports {sw[7]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
set_property PACKAGE_PIN V2 [get_ports {sw[8]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[8]}]
set_property PACKAGE_PIN T3 [get_ports {sw[9]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[9]}]
```

```
set_property PACKAGE_PIN T2 [get_ports {sw[10]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[10]}]
set_property PACKAGE_PIN R3 [get_ports {sw[11]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[11]}]
set_property PACKAGE_PIN W2 [get_ports {sw[12]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[12]}]
set_property PACKAGE_PIN U1 [get_ports {sw[13]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[13]}]
set_property PACKAGE_PIN T1 [get_ports {sw[14]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[14]}]
set_property PACKAGE_PIN R2 [get_ports {sw[15]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {sw[15]}]


# LEDs
set_property PACKAGE_PIN U16 [get_ports {led[0]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
set_property PACKAGE_PIN E19 [get_ports {led[1]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
set_property PACKAGE_PIN U19 [get_ports {led[2]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
set_property PACKAGE_PIN V19 [get_ports {led[3]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
set_property PACKAGE_PIN W18 [get_ports {led[4]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
set_property PACKAGE_PIN U15 [get_ports {led[5]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
set_property PACKAGE_PIN U14 [get_ports {led[6]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
set_property PACKAGE_PIN V14 [get_ports {led[7]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
set_property PACKAGE_PIN V13 [get_ports {led[8]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[8]}]
set_property PACKAGE_PIN V3 [get_ports {led[9]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[9]}]
set_property PACKAGE_PIN W3 [get_ports {led[10]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[10]}]
set_property PACKAGE_PIN U3 [get_ports {led[11]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[11]}]
set_property PACKAGE_PIN P3 [get_ports {led[12]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[12]}]
set_property PACKAGE_PIN N3 [get_ports {led[13]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[13]}]
set_property PACKAGE_PIN P1 [get_ports {led[14]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[14]}]
set_property PACKAGE_PIN L1 [get_ports {led[15]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {led[15]}]


#7 catment display
set_property PACKAGE_PIN W7 [get_ports {cat[0]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {cat[0]}]
set_property PACKAGE_PIN W6 [get_ports {cat[1]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {cat[1]}]
set_property PACKAGE_PIN U8 [get_ports {cat[2]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {cat[2]}]
```

```
set_property PACKAGE_PIN V8 [get_ports {cat[3]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {cat[3]}]
set_property PACKAGE_PIN U5 [get_ports {cat[4]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {cat[4]}]
set_property PACKAGE_PIN V5 [get_ports {cat[5]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {cat[5]}]
set_property PACKAGE_PIN U7 [get_ports {cat[6]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {cat[6]}]



set_property PACKAGE_PIN U2 [get_ports {an[0]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]


#Buttons
set_property PACKAGE_PIN U18 [get_ports {btn[0]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {btn[0]}]
set_property PACKAGE_PIN T18 [get_ports {btn[1]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {btn[1]}]
set_property PACKAGE_PIN W19 [get_ports {btn[2]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {btn[2]}]
set_property PACKAGE_PIN T17 [get_ports btn[3]]
 set_property IOSTANDARD LVCMOS33 [get_ports {btn[3]}]
set_property PACKAGE_PIN U17 [get_ports {btn[4]}]
 set_property IOSTANDARD LVCMOS33 [get_ports {btn[4]}]
```